

CAPSTONE PROJECT

Automated Finance Dashboard for Small Business

Financial Transactions Dataset

Python Project

[Reference Document from the Jupyter Notebook](#)

Code 1

```
import pandas as pd
```

Code 2

```
df = pd.read_csv("financial_transactions.csv")
```

Code 3

```
df
```

OUTPUT

df										
	Transaction_ID	Date	Customer	Category	Product	Quantity	Sales	Profit	Payment_Method	Region
0	1	2026-07-08	Anu	Electronics	Laptop	5	167990	21688	Net Banking	North
1	2	2026-03-01	Nikhil	Stationery	File	1	111	11	Net Banking	North
2	3	2026-05-31	Akhil	Stationery	Pen	9	8181	1051	Cash	South
3	4	2026-07-15	Ravi	Grocery	Sugar	5	3430	436	Cash	North
4	5	2026-07-26	Meena	Grocery	Rice	6	8748	1861	UPI	North
...	...	...	...	...	...	...	...	...	...	...
495	496	2026-01-20	Priya	Clothing	Jacket	8	15624	4630	Credit Card	North
496	497	2026-02-03	Priya	Grocery	Rice	7	8351	1690	Net Banking	East
497	498	2026-08-26	Ravi	Electronics	Headphones	8	174320	15477	Net Banking	North
498	499	2026-04-02	Rahul	Stationery	Marker	8	13336	2747	Debit Card	South
499	500	2026-04-17	Swathi	Clothing	Shoes	2	13882	2653	Net Banking	West

500 rows × 10 columns

Code 4

Df.shape

OUTPUT

df.shape	     
(500, 10)	

Code 5

df.head()

OUTPUT

```
: df.head()
```

	Transaction_ID	Date	Customer	Category	Product	Quantity	Sales	Profit	Payment_Method	Region
0	1	2026-07-08	Anu	Electronics	Laptop	5	167990	21688	Net Banking	North
1	2	2026-03-01	Nikhil	Stationery	File	1	111	11	Net Banking	North
2	3	2026-05-31	Akhil	Stationery	Pen	9	8181	1051	Cash	South
3	4	2026-07-15	Ravi	Grocery	Sugar	5	3430	436	Cash	North
4	5	2026-07-26	Meena	Grocery	Rice	6	8748	1861	UPI	North

Code 6

```
df.tail(3)
```

OUTPUT

```
[8]: df.tail()
```

	Transaction_ID	Date	Customer	Category	Product	Quantity	Sales	Profit	Payment_Method	Region
495	496	2026-01-20	Priya	Clothing	Jacket	8	15624	4630	Credit Card	North
496	497	2026-02-03	Priya	Grocery	Rice	7	8351	1690	Net Banking	East
497	498	2026-08-26	Ravi	Electronics	Headphones	8	174320	15477	Net Banking	North
498	499	2026-04-02	Rahul	Stationery	Marker	8	13336	2747	Debit Card	South
499	500	2026-04-17	Swathi	Clothing	Shoes	2	13882	2653	Net Banking	West

Code 7

```
df.info()
```

OUTPUT

```
[9]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Transaction_ID    500 non-null    int64
1   Date              500 non-null    object
2   Customer          500 non-null    object
3   Category          500 non-null    object
4   Product           500 non-null    object
5   Quantity          500 non-null    int64
6   Sales             500 non-null    int64
7   Profit            500 non-null    int64
8   Payment_Method    500 non-null    object
9   Region            500 non-null    object
dtypes: int64(4), object(6)
memory usage: 39.2+ KB
```

Code 8

Df.dtypes

OUTPUT

```
df.dtypes
```

```
Transaction_ID    int64
Date              object
Customer          object
Category          object
Product           object
Quantity          int64
Sales             int64
Profit            int64
Payment_Method    object
Region            object
dtype: object
```

Code 9

```
print("Missing values in each column:") print(df.isnull().sum())
```

OUTPUT

```
print("Missing values in each column:")  
print(df.isnull().sum())
```

```
Missing values in each column:  
Transaction_ID    0  
Date              0  
Customer          0  
Category          0  
Product           0  
Quantity          0  
Sales             0  
Profit            0  
Payment_Method    0  
Region            0  
dtype: int64
```

Code 10

```
df.duplicated().sum()
```

OUTPUT

```
df.duplicated().sum()
```

```
np.int64(0)
```

Code 11

```
df.describe()
```

OUTPUT

```
df.describe()
```

	Transaction_ID	Quantity	Sales	Profit
count	500.000000	500.000000	500.000000	500.000000
mean	250.500000	5.588000	77935.808000	14718.44400
std	144.481833	2.884944	122810.672351	25053.37623
min	1.000000	1.000000	111.000000	11.00000
25%	125.750000	3.000000	6823.250000	1203.75000
50%	250.500000	6.000000	20646.000000	3654.00000
75%	375.250000	8.000000	94242.000000	16099.50000
max	500.000000	10.000000	747490.000000	156830.00000

Code 12

```
print("Categories:")
```

```
print(df["Category"].value_counts())
```

OUTPUT

```
print("Categories:")  
print(df["Category"].value_counts())
```

```
Categories:  
Category  
Furniture      114  
Grocery        100  
Electronics    97  
Clothing       95  
Stationery     94  
Name: count, dtype: int64
```

Code 13

```
df["Sales"].sum()
```

```
df["Sales"].mean()
```

```
df["Profit"].mean()
```

OUTPUT

```
df["Sales"].sum()
df["Sales"].mean()
df["Profit"].mean()
np.float64(14718.444)
```

Code 14

```
text_columns = [
```

```
    "Customer",
```

```
    "Category",
```

```
    "Product",
```

```
    "Payment_Method",
```

```
    "Region"
```

```
]
```

```
for col in text_columns:
```

```
    df[col] = df[col].astype(str).str.strip()
```

```
print("Text columns cleaned successfully!")
```

OUTPUT

```

text_columns = [
    "Customer",
    "Category",
    "Product",
    "Payment_Method",
    "Region"
]

for col in text_columns:
    df[col] = df[col].astype(str).str.strip()

print("Text columns cleaned successfully!")

```

Text columns cleaned successfully!

Code 15

```
print("Negative Sales values:", (df["Sales"] < 0).sum())
```

```
print("Negative Profit values:", (df["Profit"] < 0).sum())
```

```
print("Negative Quantity values:", (df["Quantity"] < 0).sum())
```

OUTPUT

```

# Check Invalid Sales, Profit and Quantity

print("Negative Sales values:", (df["Sales"] < 0).sum())
print("Negative Profit values:", (df["Profit"] < 0).sum())
print("Negative Quantity values:", (df["Quantity"] < 0).sum())

```

```

Negative Sales values: 0
Negative Profit values: 0
Negative Quantity values: 0

```

Code 16

```
total_sales = df["Sales"].sum()
```

```
print("Total Sales Value:", total_sales)
```

OUTPUT

```
total_sales = df["Sales"].sum()
print("Total Sales Value:", total_sales)
Total Sales Value: 38967904
```

Code 17

```
average_sales = df["Sales"].mean()
print("Average Sales:", average_sales)
```

OUTPUT

```
average_sales = df["Sales"].mean()
print("Average Sales:", average_sales)
Average Sales: 77935.808
```

Code 18

```
highest_sale = df["Sales"].max()
print("Highest Transaction Sales:", highest_sale)
```

OUTPUT

```
highest_sale = df["Sales"].max()
print("Highest Transaction Sales:", highest_sale)
Highest Transaction Sales: 747400
```

Code 19

```
lowest_sale = df["Sales"].min()

print("Lowest Transaction Sales:", lowest_sale)
```

OUTPUT

```
lowest_sale = df["Sales"].min()

print("Lowest Transaction Sales:", lowest_sale)

Lowest Transaction Sales: 111
```

Code 20

```
total_transactions = len(df)

print("Total Number of Transactions:", total_transactions)
```

OUTPUT

```
total_transactions = len(df)

print("Total Number of Transactions:", total_transactions)

Total Number of Transactions: 500
```

Code 21

```
total_profit = df["Profit"].sum()

print("Total Profit:", total_profit)
```

OUTPUT

```
total_profit = df["Profit"].sum()

print("Total Profit:", total_profit)

Total Profit: 7359222
```



Code 22

```
category_summary = df.groupby("Category")["Sales"].agg(  
    Number_of_Transactions="count",  
    Total_Sales="sum",  
    Average_Sales="mean"  
).sort_values(  
    "Total_Sales",  
    ascending=False  
)
```

category_summary

OUTPUT

```
# GROUP BY Analysis - Category

category_summary = df.groupby("Category")["Sales"].agg(
    Number_of_Transactions="count",
    Total_Sales="sum",
    Average_Sales="mean"
).sort_values(
    "Total_Sales",
    ascending=False
)

category_summary
```

	Number_of_Transactions	Total_Sales	Average_Sales
Category			
Electronics	97	22256321	229446.608247
Furniture	114	13063762	114594.403509
Clothing	95	2248310	23666.421053
Grocery	100	797596	7975.960000
Stationery	94	601915	6403.351064

Code 23

```
region_summary = df.groupby("Region")["Sales"].agg(
    Number_of_Transactions="count",
    Total_Sales="sum",
    Average_Sales="mean"
).sort_values(
    "Total_Sales",
    ascending=False
)

region_summary
```

OUTPUT

```
region_summary = df.groupby("Region")["Sales"].agg(  
    Number_of_Transactions="count",  
    Total_Sales="sum",  
    Average_Sales="mean"  
).sort_values(  
    "Total_Sales",  
    ascending=False  
)  
  
region_summary
```

	Number_of_Transactions	Total_Sales	Average_Sales
Region			
West	140	13030578	93075.557143
North	123	9987729	81201.048780
South	115	8041060	69922.260870
East	122	7908537	64824.073770

Code 24

```
payment_summary = df.groupby("Payment_Method")["Sales"].agg(  
    Number_of_Transactions="count",  
    Total_Sales="sum",  
    Average_Sales="mean"  
)  
.sort_values(  
    "Total_Sales",  
    ascending=False  
)  
  
payment_summary
```

OUTPUT

```
payment_summary = df.groupby("Payment_Method")["Sales"].agg(  
    Number_of_Transactions="count",  
    Total_Sales="sum",  
    Average_Sales="mean"  
).sort_values(  
    "Total_Sales",  
    ascending=False  
)  
  
payment_summary
```

	Number_of_Transactions	Total_Sales	Average_Sales
Payment_Method			
Net Banking	113	11009381	97428.150442
Cash	99	7948257	80285.424242
UPI	112	7132282	63681.089286
Credit Card	89	6779155	76170.280899
Debit Card	87	6098829	70101.482759

Code 25

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

Code 26

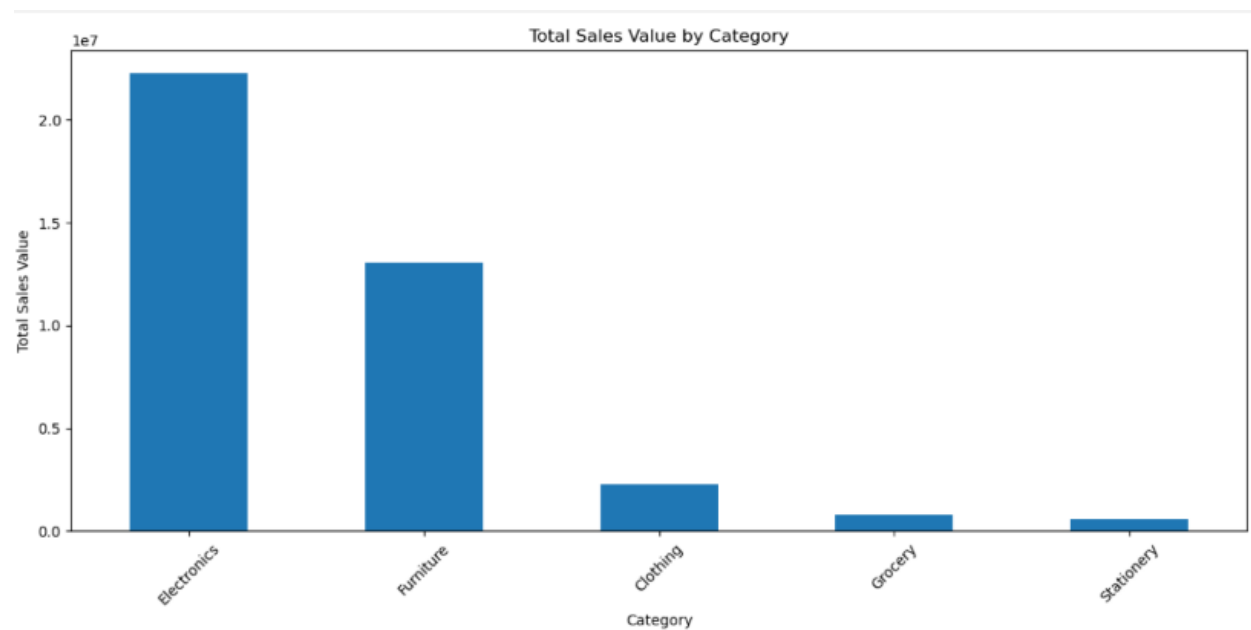
```
plt.figure(figsize=(12, 6))
```

```
category_summary["Total_Sales"].plot(  
    kind="bar"
```

```
)
```

```
plt.title("Total Sales Value by Category")
plt.xlabel("Category")
plt.ylabel("Total Sales Value")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

OUTPUT



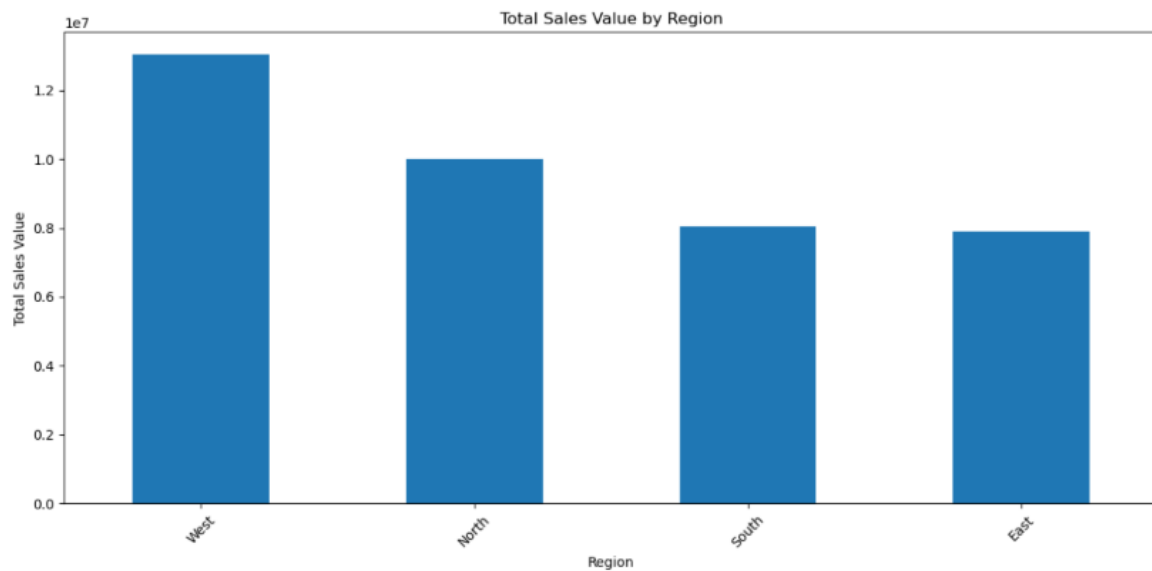
Code 27

```
plt.figure(figsize=(12, 6))

region_summary["Total_Sales"].plot(
    kind="bar"
)
```

```
plt.title("Total Sales Value by Region")
plt.xlabel("Region")
plt.ylabel("Total Sales Value")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

OUTPUT



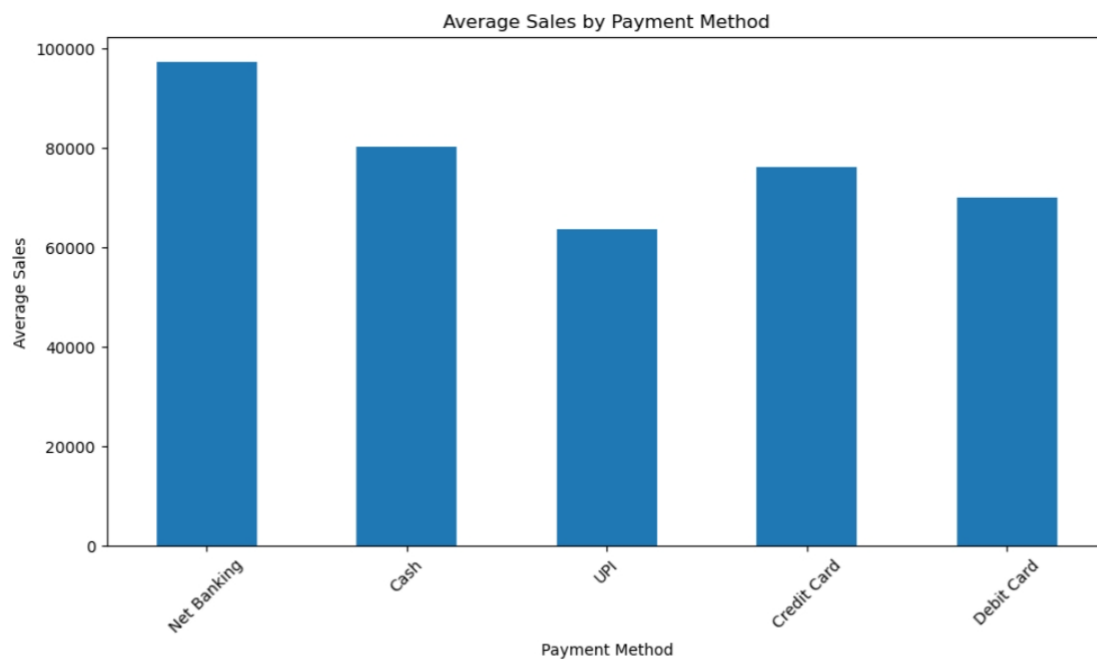
Code 28

```
plt.figure(figsize=(10, 6))

payment_summary["Average_Sales"].plot(
    kind="bar"
)
```

```
plt.title("Average Sales by Payment Method")
plt.xlabel("Payment Method")
plt.ylabel("Average Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

OUTPUT



```
df["Date"] = pd.to_datetime(df["Date"])
```

Code 29

```
df["Date"] = pd.to_datetime(df["Date"])

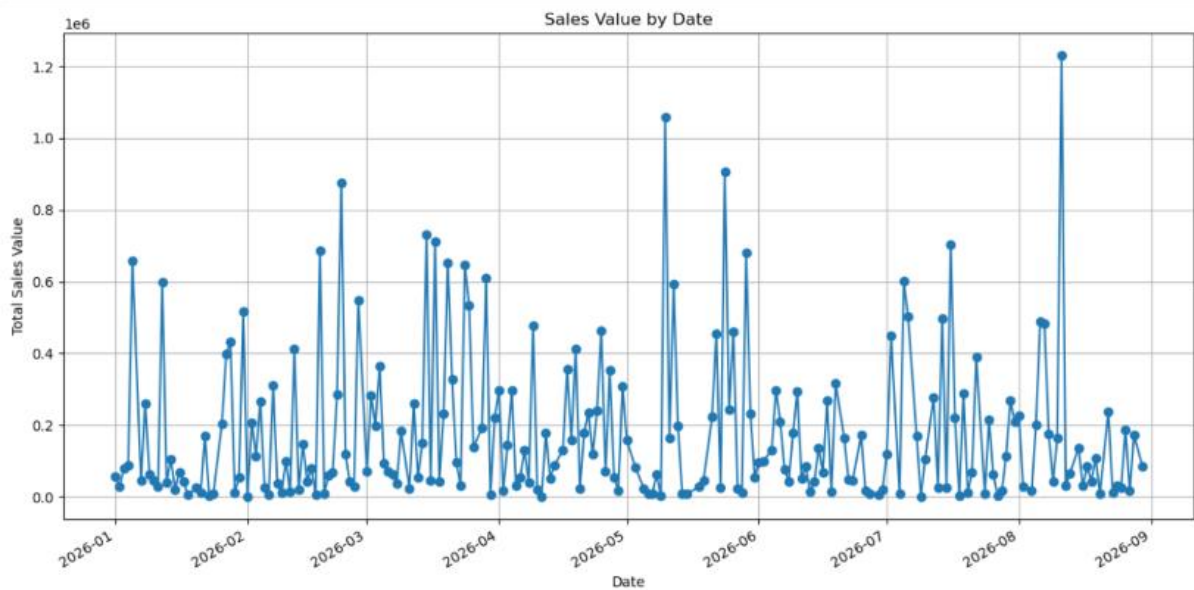
daily_sales = df.groupby("Date")["Sales"].sum()

plt.figure(figsize=(12, 6))

daily_sales.plot( kind="line", marker="o" )

plt.title("Sales Value by Date") plt.xlabel("Date") plt.ylabel("Total Sales Value")
plt.grid(True) plt.tight_layout() plt.show()
```

OUTPUT



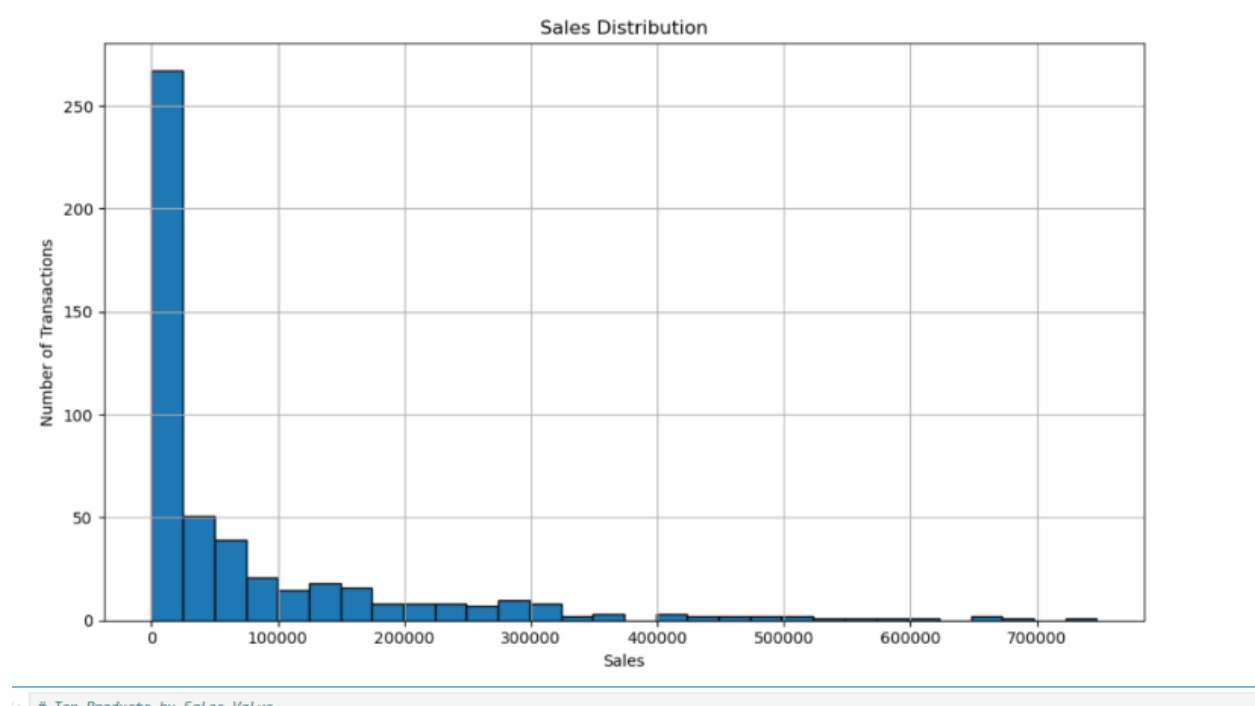
Code 30

```
plt.figure(figsize=(10, 6))

plt.hist( df["Sales"], bins=30, edgecolor="black" )

plt.title("Sales Distribution") plt.xlabel("Sales") plt.ylabel("Number of Transactions")
plt.grid(True) plt.tight_layout() plt.show()
```

OUTPUT



Code 31

```
top_products = df.groupby("Product")["Sales"] \
    .sum() \
```

```
.sort_values(ascending=False)
```

```
top_10_products = top_products.head(10)
```

```
top_10_products
```

OUTPUT

```
# Top Products by Sales Value

top_products = df.groupby("Product")["Sales"] \
    .sum() \
    .sort_values(ascending=False)

top_10_products = top_products.head(10)

top_10_products
```

```
Product
Headphones    6651629
Smartphone    5821377
Laptop         4011847
Desk           3749482
Keyboard       3210209
Bookshelf      2564317
Monitor        2561259
Chair          2367729
Sofa           2264011
Table          2118223
Name: Sales, dtype: int64
```

Code 32

```
plt.figure(figsize=(12, 6))
```

```
top_10_products.plot(
```

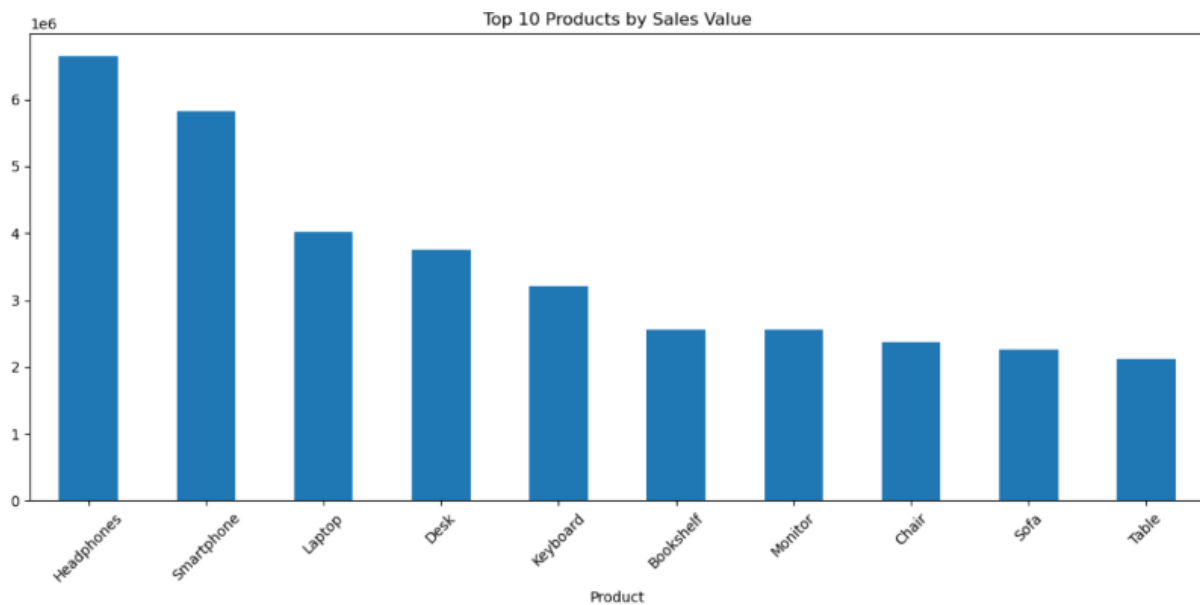
```
    kind="bar"
```

```
)
```

```
plt.title("Top 10 Products by Sales Value")
```

```
plt.xlabel("Product")
plt.ylabel("Total Sales Value")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

OUTPUT



Code 33

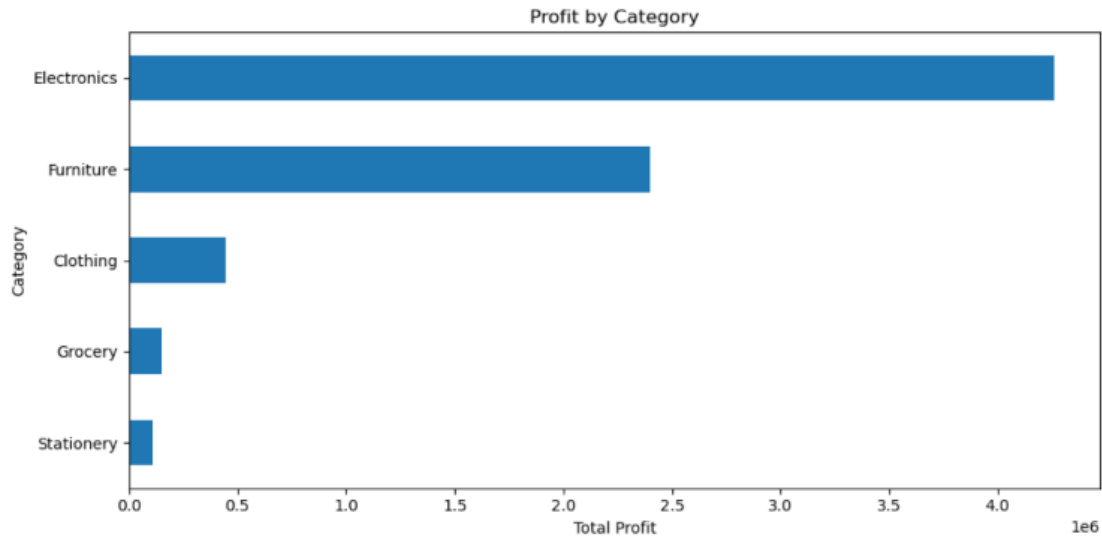
```
category_profit = df.groupby("Category")["Profit"].sum()

plt.figure(figsize=(10, 5))

category_profit.sort_values().plot( kind="barh" )

plt.title("Profit by Category") plt.xlabel("Total Profit") plt.ylabel("Category")
plt.tight_layout() plt.show()
```

OUTPUT



Code 34

```
payment_sales = df.groupby("Payment_Method")["Sales"].sum()
```

```
plt.figure(figsize=(8, 8))
```

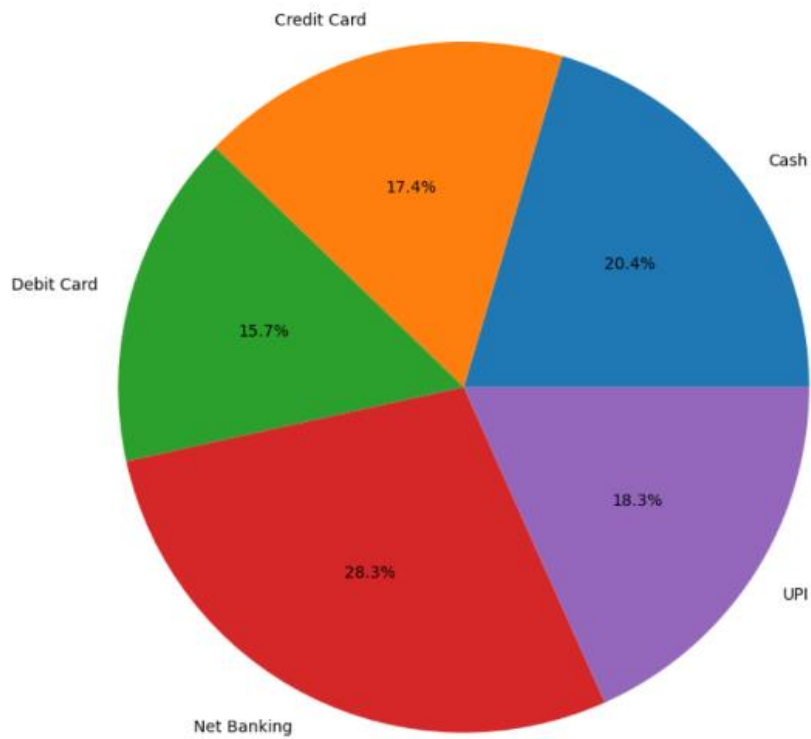
```
plt.pie(  
    payment_sales,  
    labels=payment_sales.index,  
    autopct="%1.1f%%"  
)
```

```
plt.title("Sales Distribution by Payment Method")
```

```
plt.tight_layout()
```

```
plt.show()
```

OUTPUT



Code 35

```
print("===== FINANCIAL DASHBOARD KPIs =====")
```

```
print("Total Sales Value :", total_sales)
```

```
print("Total Profit :", total_profit)
```

```
print("Average Sales :", average_sales)
```

```
print("Highest Transaction Sales :", highest_sale)
```

```
print("Lowest Transaction Sales :", lowest_sale)
```

```
print("Total Transactions :", total_transactions)
```

OUTPUT

```
print("----- FINANCIAL DASHBOARD KPIS -----")  
  
print("Total Sales Value :", total_sales)  
print("Total Profit :", total_profit)  
print("Average Sales :", average_sales)  
print("Highest Transaction Sales :", highest_sale)  
print("Lowest Transaction Sales :", lowest_sale)  
print("Total Transactions :", total_transactions)
```

```
----- FINANCIAL DASHBOARD KPIS -----  
Total Sales Value : 38967904  
Total Profit : 7359222  
Average Sales : 77935.808  
Highest Transaction Sales : 747490  
Lowest Transaction Sales : 111  
Total Transactions : 500
```

Code 36

```
top_category = category_summary["Total_Sales"].idxmax()
```

```
top_category_sales = category_summary["Total_Sales"].max()
```

```
print("Top Category:", top_category)
```

```
print("Sales Value:", top_category_sales)
```

OUTPUT

```
top_category = category_summary["Total_Sales"].idxmax()
top_category_sales = category_summary["Total_Sales"].max()
print("Top Category:", top_category)
print("Sales Value:", top_category_sales)
```

```
Top Category: Electronics
Sales Value: 22256321
```

Code 37

```
top_region = region_summary["Total_Sales"].idxmax()
top_region_sales = region_summary["Total_Sales"].max()
print("Top Region:", top_region) print("Sales Value:", top_region_sales)
```

OUTPUT

```
top_region = region_summary["Total_Sales"].idxmax()
top_region_sales = region_summary["Total_Sales"].max()
print("Top Region:", top_region)
print("Sales Value:", top_region_sales)
```

```
Top Region: West
Sales Value: 13030578
```

Code 38

```
top_profit_category = category_profit.idxmax()
top_profit_value = category_profit.max()
print("Most Profitable Category:", top_profit_category) print("Profit:", top_profit_value)
```

OUTPUT

```
top_profit_category = category_profit.idxmax()

top_profit_value = category_profit.max()

print("Most Profitable Category:", top_profit_category)
print("Profit:", top_profit_value)
```

```
Most Profitable Category: Electronics
Profit: 4258542
```

Code 39

```
df.to_csv("financial_transactions_cleaned.csv", index=False)

print("Cleaned dataset saved successfully!")
```

OUTPUT

```
; df.to_csv(
    "financial_transactions_cleaned.csv",
    index=False
)

print("Cleaned dataset saved successfully!")
```

```
Cleaned dataset saved successfully!
```

Code 40

```
final_summary = {

    "Total Transactions": total_transactions,

    "Total Sales Value": total_sales,

    "Total Profit": total_profit,

    "Average Sales": average_sales,

    "Highest Transaction Sales": highest_sale,
```

```
"Lowest Transaction Sales": lowest_sale,  
"Top Category": top_category,  
"Top Region": top_region,  
"Most Profitable Category": top_profit_category  
}
```

final_summary

OUTPUT

```
final_summary ▀ {  
  "Total Transactions": total_transactions,  
  "Total Sales Value": total_sales,  
  "Total Profit": total_profit,  
  "Average Sales": average_sales,  
  "Highest Transaction Sales": highest_sale,  
  "Lowest Transaction Sales": lowest_sale,  
  "Top Category": top_category,  
  "Top Region": top_region,  
  "Most Profitable Category": top_profit_category  
}
```

final_summary

```
{'Total Transactions': 500,  
 'Total Sales Value': np.int64(38967904),  
 'Total Profit': np.int64(7359222),  
 'Average Sales': np.float64(77935.808),  
 'Highest Transaction Sales': 747490,  
 'Lowest Transaction Sales': 111,  
 'Top Category': 'Electronics',  
 'Top Region': 'West',  
 'Most Profitable Category': 'Electronics'}
```

End of Reference

Included 40 code cells and available notebook outputs/graphs.